

Informe sobre los principios SOLID

Jorman Noriega

20 de agosto de 2026

Resumen

Este informe explica de forma sencilla los cinco principios SOLID de la programación orientada a objetos. Los ejemplos utilizan situaciones de una tienda para mostrar cómo escribir código más organizado y fácil de mantener.

1. Introducción

SOLID es un conjunto de cinco principios para diseñar programas orientados a objetos. Su objetivo es reducir la dependencia entre las clases, facilitar los cambios y evitar que el código se vuelva difícil de mantener.

Las letras de SOLID representan los siguientes principios:

- S: Single Responsibility Principle.
- O: Open/Closed Principle.
- L: Liskov Substitution Principle.
- I: Interface Segregation Principle.
- D: Dependency Inversion Principle.

2. S: Responsabilidad única

Una clase debe tener una sola responsabilidad y una sola razón para cambiar. Por ejemplo, una clase `Pedido` debería encargarse de los datos del pedido, pero no de calcular totales y guardarlos en una base de datos al mismo tiempo.

Ejemplo incorrecto:

```
class Pedido {
    void agregarProducto() { }
    void calcularTotal() { }
    void guardarEnBaseDatos() { }
}
```

La clase anterior mezcla tres responsabilidades diferentes.

Ejemplo correcto:

Una solución sencilla es separar las responsabilidades:

```
class Pedido {
    private double total;
    public double getTotal() { return total; }
}

class CalculadoraPedido {
    public double calcular(Pedido pedido) {
        return pedido.getTotal();
    }
}

class PedidoRepository {
    public void guardar(Pedido pedido) {
        System.out.println("Pedido guardado");
    }
}

public class Main {
    public static void main(String[] args) {
        Pedido pedido = new Pedido();
        CalculadoraPedido calculadora = new CalculadoraPedido();
        System.out.println(calculadora.calcular(pedido));
        new PedidoRepository().guardar(pedido);
    }
}
```

De esta forma, cada clase tiene una función clara. Si cambia la forma de guardar los pedidos, no es necesario modificar la clase `Pedido`.

3. O: Abierto/cerrado

Una clase debe estar abierta para extenderse, pero cerrada para modificarse. Esto significa que podemos agregar comportamientos nuevos sin cambiar el código que ya funciona.

Ejemplo incorrecto:

```

void pagar(String tipo, double valor) {
    if (tipo.equals("tarjeta")) { }
    else if (tipo.equals("paypal")) { }
    // Cada nuevo pago modifica esta funcion.
}

```

Ejemplo correcto: se pueden agregar diferentes formas de pago mediante una interfaz:

```

interface MetodoPago {
    void pagar(double valor);
}

class TarjetaPago implements MetodoPago {
    public void pagar(double valor) {
        System.out.println("Pago con tarjeta");
    }
}

class NequiPago implements MetodoPago {
    public void pagar(double valor) {
        System.out.println("Pago con Nequi");
    }
}

public class Main {
    public static void main(String[] args) {
        MetodoPago pago = new NequiPago();
        pago.pagar(25000);
    }
}

```

Para agregar otro método, como PayPal, se crea una nueva clase sin modificar las clases anteriores.

4. L: Sustitución de Liskov

Una clase hija debe poder utilizarse en cualquier lugar donde se espere su clase padre sin causar errores ni comportamientos inesperados.

Ejemplo incorrecto:

```

class Empleado {
    public double calcularSalario() { return 2000000; }
}

class EmpleadoMedioTiempo extends Empleado {
    public double calcularSalario() {
        throw new UnsupportedOperationException();
    }
}

// Esta llamada provoca un error:
mostrarSalario(new EmpleadoMedioTiempo());

```

El empleado de medio tiempo no puede sustituir a `Empleado`, porque rompe el comportamiento esperado. Esto viola LSP.

Ejemplo correcto: los dos tipos de empleado pueden utilizarse como un `Empleado`:

```

abstract class Empleado {
    public abstract double calcularSalario();
}

class EmpleadoTiempoCompleto extends Empleado {
    public double calcularSalario() { return 2000000; }
}

class EmpleadoMedioTiempo extends Empleado {
    public double calcularSalario() { return 1000000; }
}

public class Main {
    public static void main(String[] args) {
        mostrarSalario(new EmpleadoTiempoCompleto());
        mostrarSalario(new EmpleadoMedioTiempo());
    }

    static void mostrarSalario(Empleado empleado) {
        System.out.println(empleado.calcularSalario());
    }
}

```

Una función que reciba un `Empleado` puede trabajar con cualquiera de las dos clases hijas sin conocer sus detalles.

5. I: Segregación de interfaces

Una clase no debe estar obligada a implementar métodos que no necesita. Para esto, es mejor tener varias interfaces pequeñas en lugar de una interfaz muy grande.

Ejemplo incorrecto: una interfaz grande obliga al robot a implementar métodos que no puede usar.

```

interface Trabajador {
    void trabajar();
    void comer();
    void descansar();
}

class Robot implements Trabajador {
    public void trabajar() { }
    public void comer() { /* No aplica */ }
    public void descansar() { /* No aplica */ }
}

```

Ejemplo correcto: se divide la interfaz grande en interfaces pequeñas.

```

interface Trabajable { void trabajar(); }
interface Comible { void comer(); }
interface Descansable { void descansar(); }

class Empleado implements Trabajable, Comible, Descansable {
    public void trabajar() { System.out.println("Empleado trabaja"); }
    public void comer() { System.out.println("Empleado come"); }
    public void descansar() { System.out.println("Empleado descansa"); }
}

class Robot implements Trabajable {
    public void trabajar() { System.out.println("Robot trabaja"); }
}

public class Main {
    public static void main(String[] args) {
        new Empleado().comer();
        new Robot().trabajar();
    }
}

```

El robot solo implementa Trabajable; no depende de Comible ni de Descansable.

6. D: Inversión de dependencias

Las clases principales deben depender de abstracciones, como interfaces, y no directamente de clases concretas.

Ejemplo incorrecto: el servicio depende directamente de una clase concreta.

```
class PedidoService {
    void confirmarPedido() {
        new EmailNotificador().enviar("Pedido confirmado");
    }
}
```

Ejemplo correcto: el servicio puede recibir cualquier tipo de notificador:

```
interface Notificador {
    void enviar(String mensaje);
}

class EmailNotificador implements Notificador {
    public void enviar(String mensaje) {
        System.out.println("Correo: " + mensaje);
    }
}

class PedidoService {
    private Notificador notificador;

    public PedidoService(Notificador notificador) {
        this.notificador = notificador;
    }

    public void confirmarPedido() {
        notificador.enviar("Pedido confirmado");
    }
}

public class Main {
    public static void main(String[] args) {
        Notificador notificador = new EmailNotificador();
        PedidoService servicio = new PedidoService(notificador);
        servicio.confirmarPedido();
    }
}
```

Así, `PedidoService` puede trabajar con un notificador por correo, WhatsApp u otro medio sin modificar su código.

7. Conclusión

Los principios SOLID ayudan a construir programas más ordenados, flexibles y fáciles de probar. No es necesario aplicar soluciones complejas: separar responsabilidades, usar interfaces y evitar dependencias directas ya mejora considerablemente la calidad del código.

En el proyecto de ejemplo, los cinco principios se aplican a diferentes partes de una tienda: pedidos, pagos, empleados, impresoras y notificaciones.